DESIGN AND ANALYSIS OF ALGORITHMS
Assignment 2 (CO2)

Brute-Force and Exhaustive-Search Techniques: Predict, Design, Implement, Measure, Analyse, Redesign

Name: Natesa Krishnan V

Registration number: 192511311 (Group A)

Nature of assignment: Individual

Q1 - Sorting Under Different Data Conditions

Prediction and reasoning (before implementation)

- Selection sort: it scans the whole unsorted suffix on every pass, so its comparisons are always n(n-1)/2, whatever the input. It performs at most n-1 swaps, because it swaps only once per pass.
- Bubble sort (with the standard early-exit flag): every adjacent swap removes exactly one inversion, so swaps always equal the number of inversions. The early-exit flag stops it after one pass on sorted input. On nearly sorted input it finishes after a small number of passes; on reversed input it runs almost the full n(n-1)/2 comparisons.
- Insertion sort: each element shifts left past every larger element before it, so element shifts equal the inversion count. Its comparisons are the inversions plus one failing comparison for each element that does not reach the front. It should be close to linear on nearly sorted input.
- Prediction for the decision: insertion sort should be the recommended choice for a feed that is about 95% in order, because its cost tracks the number of out-of-order pairs rather than n squared.

Three deliberately designed input conditions

- Contrast 1 - nearly sorted (about 95% in order): start from sorted keys and locally displace 5% of positions. The measured adjacent-in-order rate is 93-94%, close to the scenario's 95%.
- Contrast 2 - reversed (worst case): every pair is an inversion, n(n-1)/2 inversions in total.
- Edge case - all keys equal (heavy duplication, sorted already): zero inversions, and no key is ever out of order. Random input is included as a reference.
-

Results (n = 1,000 and n = 2,000)

Dataset	Algorithm	Comp. (n=1000)	Swaps/moves (n=1000)	Inversions	Comp. (n=2000)	Swaps/moves (n=2000)	Time ms (n=2000)
nearly_sorted_95pct	Selection	499,500	43	107	1,999,000	86	127.514
nearly_sorted_95pct	Bubble	3,990	107	107	7,990	262	0.564
nearly_sorted_95pct	Insertion	1,106	178	107	2,261	433	0.267
random	Selection	499,500	994	250,228	1,999,000	1,995	119.093
random	Bubble	499,445	250,228	250,228	1,997,515	1,014,005	254.363
random	Insertion	251,223	251,221	250,228	1,015,993	1,015,997	132.375
reversed	Selection	499,500	500	499,500	1,999,000	1,000	125.019
reversed	Bubble	499,500	499,500	499,500	1,999,000	1,999,000	310.656
reversed	Insertion	499,500	500,499	499,500	1,999,000	2,000,999	273.383
all_equal_edge	Selection	499,500	0	0	1,999,000	0	121.509
all_equal_edge	Bubble	999	0	0	1,999	0	0.128
all_equal_edge	Insertion	999	0	0	1,999	0	0.222

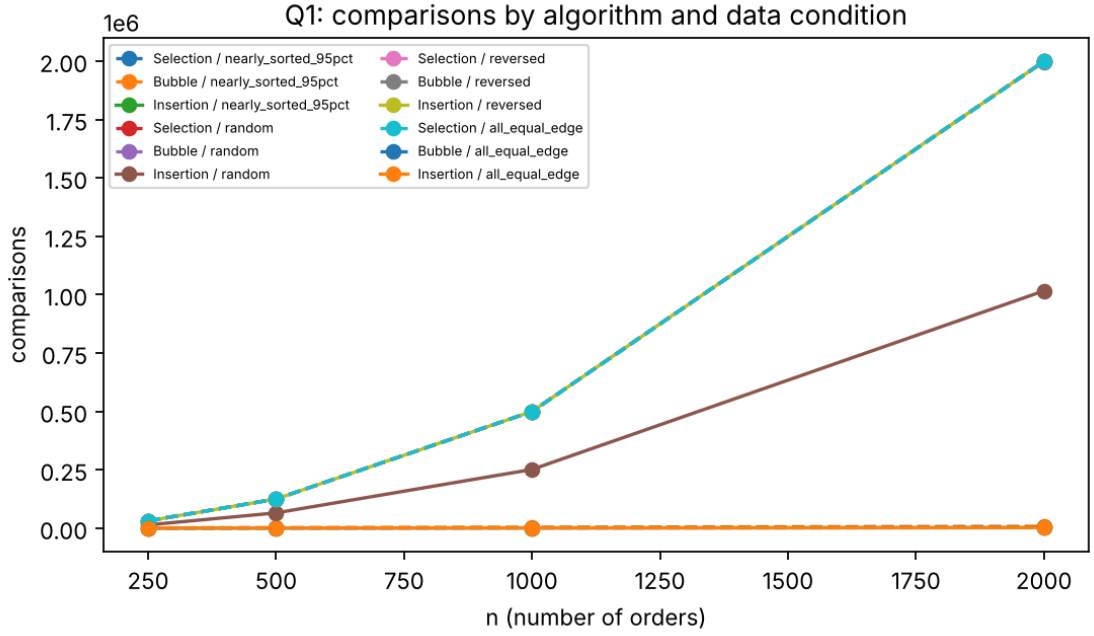


Figure 1. Comparisons against n for each algorithm and data condition.

Comparison of prediction and observation

- Selection sort: 499,500 comparisons for n = 1,000 on every dataset, exactly as predicted. Swaps stayed at or below n-1 (for example 43 on nearly sorted and 994 on random).
- Bubble swaps equal the inversion count exactly on every dataset (250,228; 499,500; 107; 0). Bubble's early-exit flag stops after one pass on all-equal input (999 comparisons = n-1), and on nearly sorted input it needs 3,990 comparisons instead of 499,500.
- Insertion moves are shifts plus key placements; the shifts match the inversion count, so moves exceed inversions only by the placements (for example, 251,221 moves against 250,228 inversions on random n = 1,000, a difference of 993 placements). Its comparisons match inversions + (n-1) minus the number of elements that reach the front: 107 + 999 = 1,106 on nearly sorted, 0 + 999 = 999 on all-equal, and exactly 499,500 on reversed, where every element reaches the front and no failing comparison occurs.
- Timing follows the counts: insertion sort took 0.27 ms on nearly sorted input at n = 2,000, against 0.56 ms for bubble sort and 127.5 ms for selection sort.

Recommendation for the fulfilment centre

Insertion sort is the recommended algorithm. About 95% of orders are already in priority order and new batches are appended periodically, so the number of out-of-order pairs stays small. The measured cost of insertion sort follows the number of inversions, which the operating profile keeps low. At n = 2,000 on nearly sorted data it needs 2,261 comparisons, against 7,990 for bubble sort and 1,999,000 for selection sort. Selection sort should not be used, because its cost ignores the input order. The choice depends on the data: on random input the three algorithms are of the same order, and on reversed input insertion sort does the most data movement (2,000,999 moves at n = 2,000). The decision is therefore not 'always insertion sort', but 'insertion sort for this data profile, re-checked if the profile changes'.

Output :


IDLE Shell 3.13.11
File Edit Shell Debug Options Window Help
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Nadesa Krishnan\Downloads\DAA_A2_192511311_Natesa_Krishnan_V\DAA-A2-192511311-Natesa-Krishnan-V\src\q1_sorting.py
{'dataset': 'nearly_sorted_95pct', 'n': 500, 'algorithm': 'Selection', 'comparisons': 124750, 'swaps_or_moves': 20, 'time_ms': 8.6363, 'inversions': 46, 'adjacent_in_order_pct': 94.59}
{'dataset': 'nearly_sorted_95pct', 'n': 500, 'algorithm': 'Bubble', 'comparisons': 2485, 'swaps_or_moves': 46, 'time_ms': 0.1966, 'inversions': 46, 'adjacent_in_order_pct': 94.59}
{'dataset': 'nearly_sorted_95pct', 'n': 500, 'algorithm': 'Insertion', 'comparisons': 545, 'swaps_or_moves': 76, 'time_ms': 0.0791, 'inversions': 46, 'adjacent_in_order_pct': 94.59}
{'dataset': 'random', 'n': 500, 'algorithm': 'Selection', 'comparisons': 124750, 'swaps_or_moves': 495, 'time_ms': 6.3711, 'inversions': 64751, 'adjacent_in_order_pct': 51.3}
{'dataset': 'random', 'n': 500, 'algorithm': 'Bubble', 'comparisons': 124254, 'swaps_or_moves': 64751, 'time_ms': 11.1369, 'inversions': 64751, 'adjacent_in_order_pct': 51.3}
{'dataset': 'random', 'n': 500, 'algorithm': 'Insertion', 'comparisons': 65239, 'swaps_or_moves': 65244, 'time_ms': 6.1992, 'inversions': 64751, 'adjacent_in_order_pct': 51.3}
{'dataset': 'reversed', 'n': 500, 'algorithm': 'Selection', 'comparisons': 124750, 'swaps_or_moves': 250, 'time_ms': 5.7939, 'inversions': 124750, 'adjacent_in_order_pct': 0.0}
{'dataset': 'reversed', 'n': 500, 'algorithm': 'Bubble', 'comparisons': 124750, 'swaps_or_moves': 124750, 'time_ms': 13.3768, 'inversions': 124750, 'adjacent_in_order_pct': 0.0}
{'dataset': 'reversed', 'n': 500, 'algorithm': 'Insertion', 'comparisons': 124750, 'swaps_or_moves': 125249, 'time_ms': 13.6472, 'inversions': 124750, 'adjacent_in_order_pct': 0.0}
{'dataset': 'all_equal_edge', 'n': 500, 'algorithm': 'Selection', 'comparisons': 124750, 'swaps_or_moves': 0, 'time_ms': 5.5668, 'inversions': 0, 'adjacent_in_order_pct': 100.0}
{'dataset': 'all_equal_edge', 'n': 500, 'algorithm': 'Bubble', 'comparisons': 499, 'swaps_or_moves': 0, 'time_ms': 0.0244, 'inversions': 0, 'adjacent_in_order_pct': 100.0}
{'dataset': 'all_equal_edge', 'n': 500, 'algorithm': 'Insertion', 'comparisons': 499, 'swaps_or_moves': 0, 'time_ms': 0.0448, 'inversions': 0, 'adjacent_in_order_pct': 100.0}
>>>

Q2 - When Is Sequential Search an Appropriate Engineering Choice?

Experimental design

Each record holds a Tool ID, Technician ID, Issue Time and Work Bay. The list is shuffled, as issue order is arbitrary. Searches are run for four designed positions (first, middle, last and absent), and for 200 random successful and 200 random unsuccessful lookups at each list size. Comparisons are counted as key equality tests. List sizes are 100, 1,000, 10,000 and 100,000 records.

n	Case	Position found	Comparisons	Time (us)
100	success_first	0	1	-
100	success_middle	50	51	-
100	success_last	99	100	-
100	unsuccessful	-1	100	-
100	random_successful_avg	-	52.67	4.719
100	unsuccessful_avg	-	100	6.127
1,000	success_first	0	1	-
1,000	success_middle	500	501	-
1,000	success_last	999	1,000	-
1,000	unsuccessful	-1	1,000	-
1,000	random_successful_avg	-	534.11	42.88
1,000	unsuccessful_avg	-	1,000	78.91
10,000	success_first	0	1	-
10,000	success_middle	5,000	5,001	-
10,000	success_last	9,999	10,000	-
10,000	unsuccessful	-1	10,000	-
10,000	random_successful_avg	-	4,897.68	518.753
10,000	unsuccessful_avg	-	10,000	991.085
100,000	success_first	0	1	-
100,000	success_middle	50,000	50,001	-
100,000	success_last	99,999	100,000	-
100,000	unsuccessful	-1	100,000	-
100,000	random_successful_avg	-	49,271.24	11,075.91
100,000	unsuccessful_avg	-	100,000	24,222.814

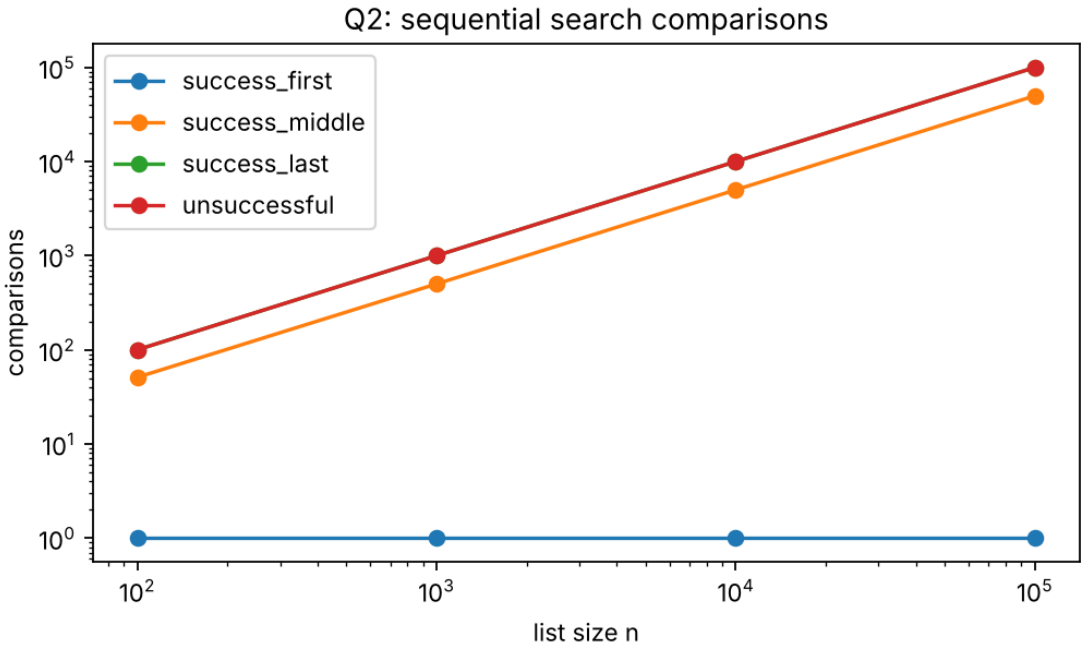


Figure 2. Sequential search comparisons (log-log) for each case.

Prediction and observed behaviour

- A successful search at position p (1-based) takes exactly p comparisons. Observed: 1, n/2+1 and n for first, middle and last positions at every size.
- An unsuccessful search always scans all n records: exactly n comparisons (100, 1,000, 10,000, 100,000 observed).
- The average successful search is (n+1)/2 comparisons. Observed averages were 52.7 at n = 100 (predicted 50.5), 534 at n = 1,000 (predicted 500.5) and 49,271 at n = 100,000 (predicted 50,000.5). The gap is consistent with sampling error from 200 random lookups.
- Time per lookup grows linearly, from about 4.7 us at n = 100 to 11,076 us at n = 100,000 for successful searches, confirming the linear cost model.

Operating boundary

I define an acceptable response time of 1 ms per lookup for this interactive tool, measured on the reference environment. Using the measured averages, an unsuccessful full scan reaches that limit at about 10,000 records (991 us observed), and a successful search reaches it at about 20,000 records (519 us at 10,000). Frequency also matters: at 10,000 records a worst-case lookup occupies roughly one millisecond of CPU, so if requests arrive faster than about 1,000 per second, one sequential list cannot keep up on one core.

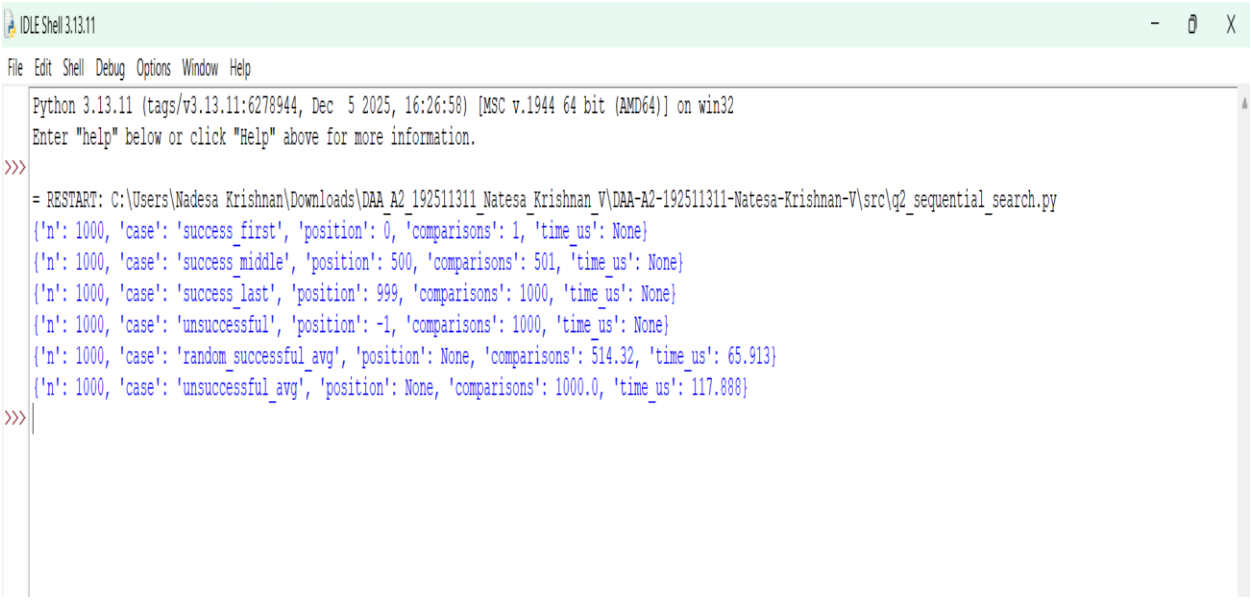
Sequential search is therefore a reasonable choice for a hangar-sized list of a few thousand records searched occasionally. I would stop recommending it once the list passes about 10,000 records, or once lookups arrive at a rate that makes the average cost per second exceed the budget.

Re-evaluation: 20 maintenance facilities

Expanding to 20 facilities multiplies the active record count and the request rate. If all records are held in one list at 10,000 records per facility (200,000 in total), a single unsuccessful scan would take about 48 ms by linear extrapolation from the measured 24 ms at 100,000 records. Each lookup is 20 times slower, and with 20 times as many requests the total work is about 400 times larger. The original recommendation no longer holds for one global list.

Two changes follow from the evidence. First, keep one list per facility, so each lookup still scans about 10,000 records and stays within budget, and route cross-facility queries by facility ID. Second, for the global 'who holds tool T-184' question, a hash index on Tool ID would give near-constant lookups. The hash index is a design proposal that I did not measure in this assignment, so I make no performance claim for it.

Output :



Q3 - Engineering the Behaviour of Brute-Force String Matching

Test conditions and predictions

- Test 1 - low work: random text over the letters a to t, and pattern 'zxqvjk' (letter z never occurs in the text). The pattern's first letter almost never occurs in the text, so most alignments fail on the first comparison. Prediction: about one comparison per alignment, roughly n in total.
- Test 2 - engineered worst case: text 'a' repeated n times, pattern 'a' repeated 99 times followed by 'b'. Every alignment matches 99 characters before failing on the last one. Prediction: (n-m+1)*m comparisons, about 100 per text character.
- Test 3 - additional condition: binary random text (alphabet {a, b}) with a 12-character pattern copied from the middle of the text. Prediction: roughly 2 comparisons per alignment before the first match, because partial matches of length 1 are common over a two-letter alphabet, and the search stops at the first match.

Results

Case	n	m	Alignments	Match index	Char comparisons	Per text char	(n-m+1)*m
low_work_random26	1,000	6	995	-1	995	0.995	5,970
worst_case_a_runs	1,000	100	901	-1	90,100	90.1	90,100
binary_random_match_mid	1,000	12	989	500	1,022	1.022	11,868
low_work_random26	2,000	6	1,995	-1	1,995	0.998	11,970
worst_case_a_runs	2,000	100	1,901	-1	190,100	95.05	190,100
binary_random_match_mid	2,000	12	1,989	1,000	2,120	1.06	23,868
low_work_random26	4,000	6	3,995	-1	3,995	0.999	23,970
worst_case_a_runs	4,000	100	3,901	-1	390,100	97.525	390,100
binary_random_match_mid	4,000	12	3,989	1,824	3,673	0.918	47,868
low_work_random26	8,000	6	7,995	-1	7,995	0.999	47,970
worst_case_a_runs	8,000	100	7,901	-1	790,100	98.763	790,100
binary_random_match_mid	8,000	12	7,989	670	1,362	0.17	95,868

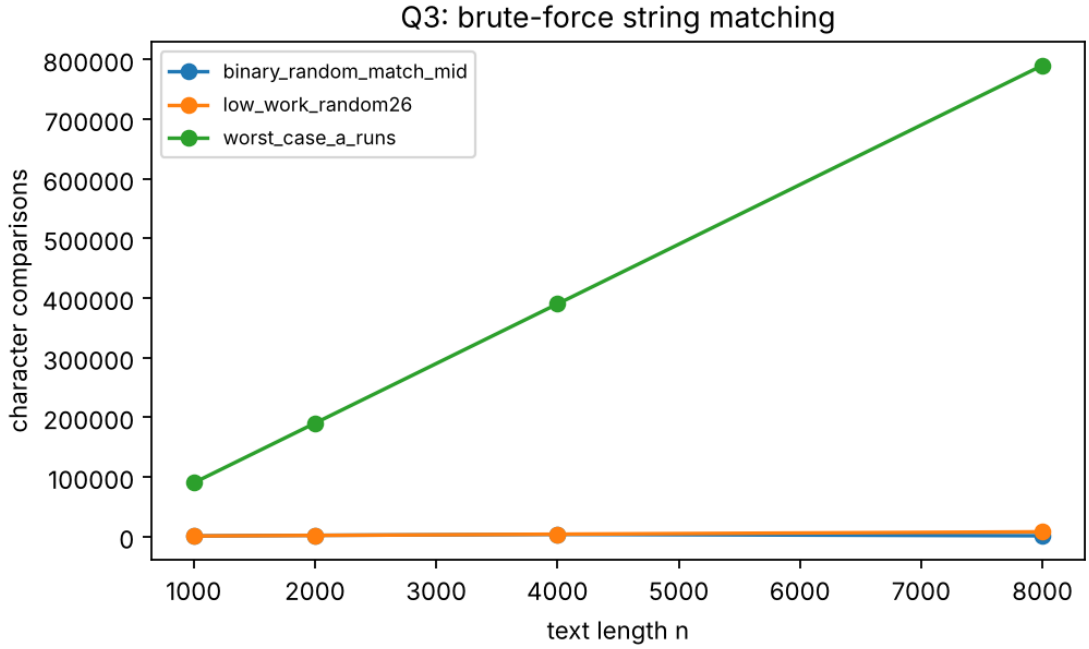


Figure 3. Character comparisons against text length for the three test conditions.

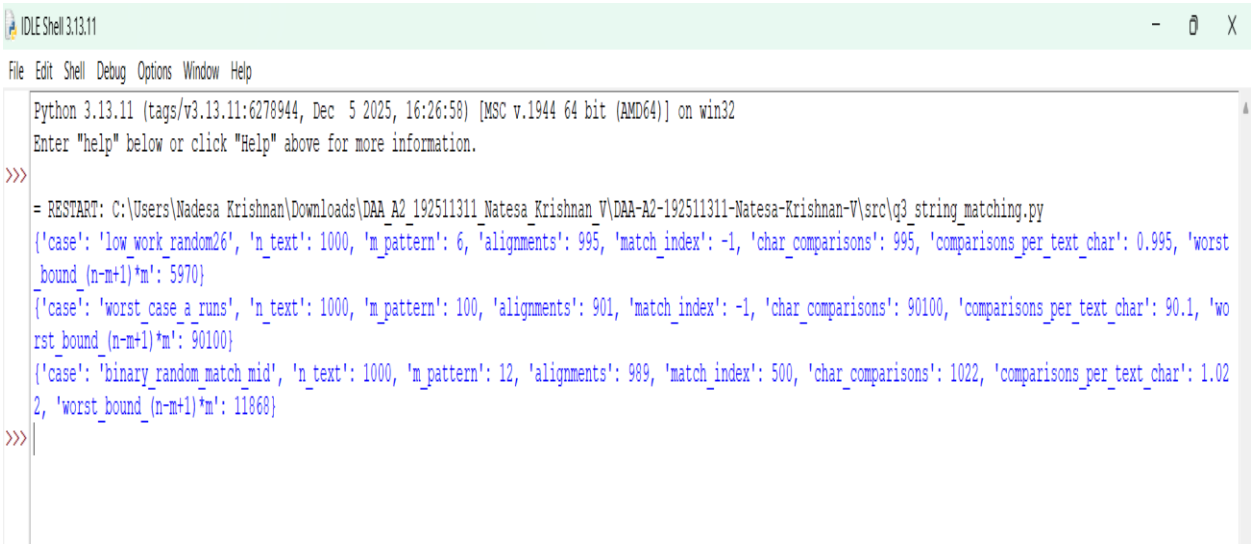
Comparison and explanation

- Test 1 matched the prediction: 995 comparisons for 995 alignments at n = 1,000, and 7,995 for 7,995 alignments at n = 8,000.
- Test 2 matched the formula exactly: 90,100 = 901 x 100 at n = 1,000 and 790,100 = 7,901 x 100 at n = 8,000. The text length alone (n = 8,000) understates the work by a factor of about 100, which is the pattern length m.
- Test 3 stopped at the first occurrence. The copied pattern appeared earlier than its source (at index 500, 1,000, 1,824 and 670 for the four sizes), and the comparison counts (1,022, 2,120, 3,673 and 1,362) are close to 2 x (match index), as predicted.

Why one input needs far more work. Work depends on how much a failed alignment has already matched before it fails. In Test 2, every alignment matches a long prefix before failing at the last character, so work is repeated across almost every alignment. In Test 1, the first letter fails immediately. The characteristics that create repeated comparisons are a pattern whose prefix repeats within the text and a small effective alphabet, combined with a text that matches that prefix at many alignments.

What this reveals. Expressing cost only as a function of input size n is misleading for this algorithm. For the same n = 8,000 text, the cost varies from about 8,000 to 790,100 comparisons, a factor of about 100, depending on the pattern. A useful analysis must include m and the structure of the text.

Output :



Q4 - Closest Pair and the Cost of Examining Every Possibility

Design, pair-count derivation and prediction

Each drone position is a point (x, y) with integer coordinates. The brute-force solution compares every unordered pair {i, j} with i < j exactly once, so no pair is examined twice, and each candidate needs one squared-distance calculation. For n drones the number of unique pairs is C(n, 2) = n(n-1)/2: drone 1 forms a new pair with each of the other n-1 drones, drone 2 forms new pairs with the n-2 drones after it (its pair with drone 1 is already counted), and so on, giving (n-1) + (n-2) + ... + 1 = n(n-1)/2. Squared distances avoid square roots and keep the comparisons exact.

Prediction: the number of pairs examined and of distance calculations should both equal n(n-1)/2 exactly, and time should grow with the square of n, so doubling n should multiply time by about four.

Results

n	n(n-1)/2	Pairs examined	Distance calcs	Closest distance	Time (ms)
100	4,950	4,950	4,950	332.543	1.129
200	19,900	19,900	19,900	504.748	4.182
400	79,800	79,800	79,800	255.227	18.075
800	319,600	319,600	319,600	151.198	72.409
1,600	1,279,200	1,279,200	1,279,200	28.653	321.352

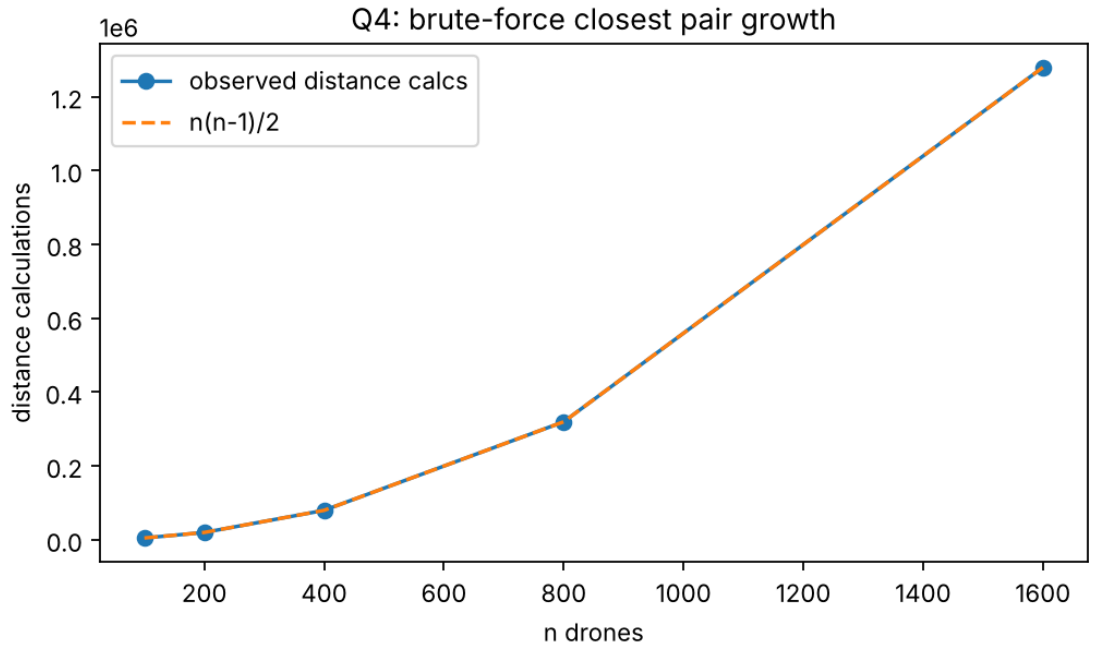


Figure 4. Distance calculations against n, observed and n(n-1)/2.

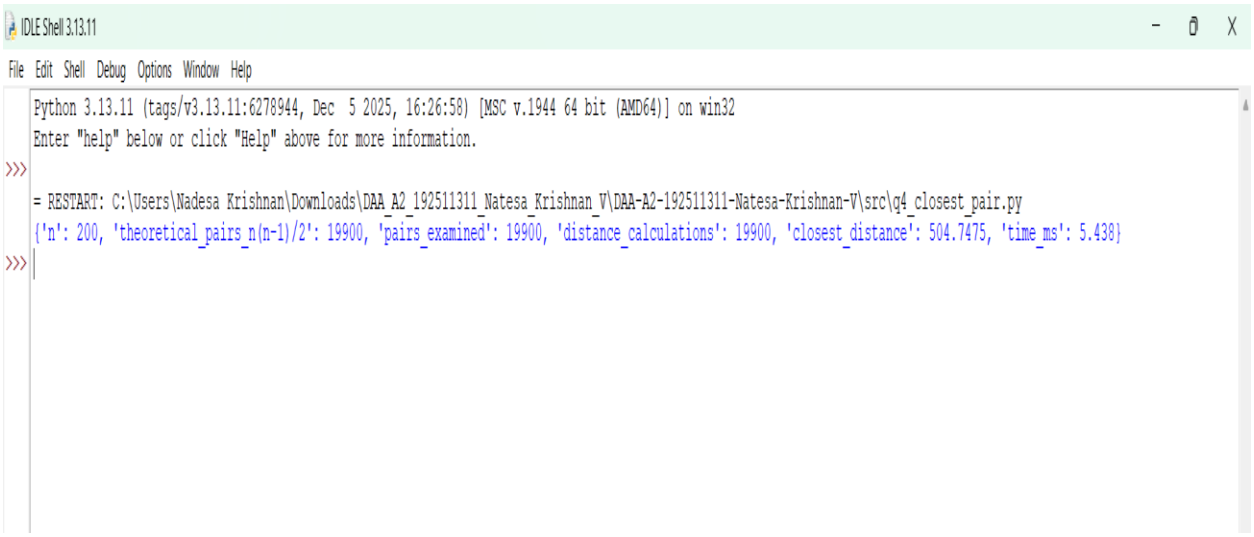
Scalability inquiry

Acceptance criterion: a full closest-pair answer should be returned within 1 second on the reference environment, which is fast enough for a collision-monitoring cycle that re-runs the check interactively.

Growth check: each doubling of n multiplies the time by about 3.7 to 4.4 (1.1 ms at n = 100, 18.1 ms at n = 400, 321 ms at n = 1,600), confirming the quadratic prediction. Each pair costs about 0.25 us on the reference machine. A 1-second budget allows about 4 million pairs, which corresponds to roughly 2,800 drones. The brute-force approach is therefore impractical above about 2,800 drones on this environment, and even more so on slower hardware or with a shorter response requirement.

Source of inefficiency. The algorithm computes the distance for every pair, including pairs that are far apart and can never be the closest. The work is also independent of the spatial arrangement: it does not exploit any ordering of the points. A redesigned strategy must avoid computing distances for pairs that cannot beat the current best, which is the approach taken in Q7.

Output :



Q5 - Deriving a Convex Hull Algorithm from a Geometric Principle

Student-derived decision rule and pseudocode

Given the principle that a line through two points can form part of the outer boundary when all other points lie on one side of it, I convert it into the following steps.

- A line through candidate points i and j is the directed segment i -> j. Using the sign of cross(i, j, k) = (xj - xi)(yk - yi) - (yj - yi)(xk - xi) gives the side of point k. A positive value means k is to the left of i -> j, a negative value means it is to the right, and zero means it is on the line.
- Because the coordinates are integers, the sign test is exact; no floating-point tolerance is needed.
- For each directed pair (i, j), examine every other point k. If any k is strictly to the right, reject i -> j immediately (early exit).
- A point exactly on the line must be handled. If it lies inside the closed segment between i and j, it does not change the edge. If it lies outside the segment, the segment is only part of a longer edge, so I reject it and let the extreme pair define the edge.
- If no point is to the right and no collinear point lies outside the segment, then i -> j is a hull edge, with the interior on its left. Recording next[i] = j for each such edge gives a counter-clockwise cycle.
- Construction: start at any vertex and follow next[] until returning to the start. The result is the hull in counter-clockwise order.

```
BRUTE_FORCE_HULL(P):
  for each ordered pair (i, j), i != j:
    is_edge <- true
    for each k != i, j:
      c <- cross(P[i], P[j], P[k])
      if c < 0: is_edge <- false; break          # k is to the right
      if c = 0 and P[k] not in closed segment [P[i], P[j]]:
        is_edge <- false; break                  # collinear, beyond the segment
    if is_edge: next[i] <- j
  start at any i with next[i] defined; follow next[] until back at start
  return the visited points (counter-clockwise)
```

Test configurations

Configuration A (mixed boundary and interior): 30 random integer points. The brute-force hull has 9 vertices and used 3,134 orientation tests. Configuration B (challenge): the bottom edge contains two collinear points, (4,0) and (8,0), which must be absorbed by the extreme pair (0,0)-(12,0) rather than reported as separate edges; the interior points (3,2), (9,3) and (6,6) sit near the boundary. Configuration B has 5 hull vertices and used 331 tests. Both results match an independent monotone-chain hull on the same points.

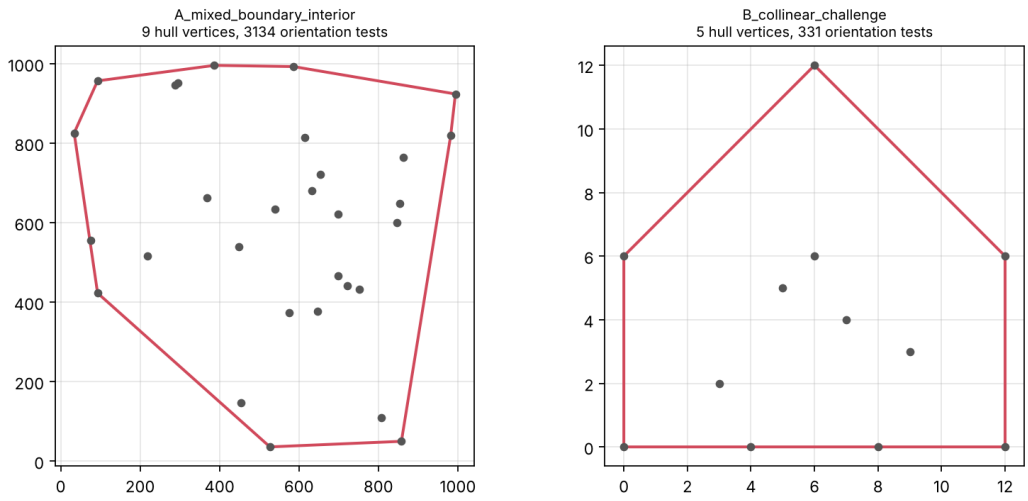


Figure 5. Boundaries produced by the brute-force method for configurations A and B.

Computational growth

n	Hull vertices	Candidate pairs	Orientation tests	Matches reference	Time (ms)
10	8	90	238	True	0.086
20	8	380	1,086	True	0.348
40	9	1,560	6,323	True	1.959
60	13	3,540	11,391	True	3.59
80	15	6,320	23,281	True	7.253
100	11	9,900	29,109	True	9.66

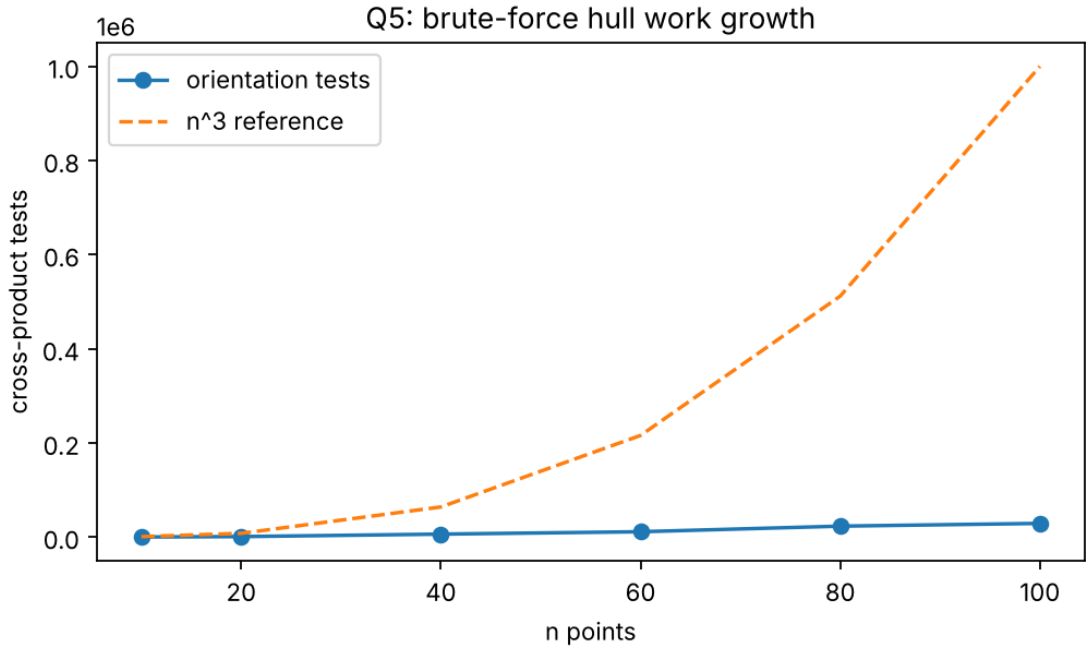


Figure 6. Orientation tests against n, with an n^3 reference curve.

The candidate pairs grow as n(n-1), so pair enumeration alone is quadratic. The worst-case work is n(n-1)(n-2), about n^3. Observed tests grew from 238 at n = 10 to 29,109 at n = 100, a factor of 122, which is closer to n^2.1 than n^3. The reason is early rejection: most directed pairs find a point to the right after only a few tests (about 3 tests per candidate pair on average). The full n-2 scan is paid only by the hull edges, of which there are 8 to 15 in these random sets, so the total work is governed by the pair count plus the number of hull edges times n. The empirical growth is therefore an observation about random inputs, while n^3 remains the worst case.

Accepted edge belongs to the outer boundary

Take a directed segment i -> j accepted by the test. Every other point lies to the left of, or on, the line through i and j, so the whole set lies in one closed half-plane bounded by that line. A line with all points on one side is a supporting line of the convex hull, and it touches the hull along a segment. Since the endpoints i and j are themselves points of the set, the touched segment is an edge of the hull, and the collinear-outside check ensures that the segment is the full edge rather than a part of a longer one. Because the points on the line are all within [i, j], both ends are genuine hull vertices. The argument uses only the sign of integer cross products, not any program output.

Output :



Q6 - Exhaustive Search and Combinatorial Growth

Exhaustive-search design

- Representation: a candidate is an n-bit integer mask. Bit i set means package i is loaded.
- Generation: masks 0 to 2^n - 1 enumerate every subset exactly once.
- Feasibility: the total weight of the selected packages must not exceed the payload limit.
- Evaluation: the total humanitarian value of the selected packages.
- Retention: keep the feasible mask with the highest value seen so far.

Correctness check on a hand-verifiable instance

Five packages A to E have (weight, value) of A(2,3), B(3,4), C(4,5), D(5,8), E(9,10), and the payload limit is 10. Checking by hand: {A, B, D} weighs 2+3+5 = 10 and is worth 3+4+8 = 15. The next best feasible options are {C, D} (weight 9, value 13), {A, B, C} (weight 9, value 12), {B, D} (weight 8, value 12) and {E} alone (value 10), so 15 is the optimum. Exhaustive search enumerated 32 subsets, of which 14 were feasible, and returned exactly {A, B, D} with value 15. A dynamic-programming solution also gives 15.

Growth prediction and experiment

Prediction: the number of candidate combinations is 2^n, so adding one package doubles the solution space and adding ten multiplies it by 1,024. Each candidate also costs about n weight checks, so total work grows as n x 2^n. Running time should therefore roughly double with each additional package.

Instance	n	Candidates	Feasible	Best value	DP check	Time (ms)
hand_5_items	5	32	14	15	15	-
random_8	8	256	80	86	86	0.226
random_10	10	1,024	294	134	134	1.001
random_12	12	4,096	1,102	183	183	4.796
random_14	14	16,384	3,996	210	210	22.913
random_16	16	65,536	15,777	310	310	110.256
random_18	18	262,144	61,554	371	371	469.49
random_20	20	1,048,576	228,150	357	357	2,122.486

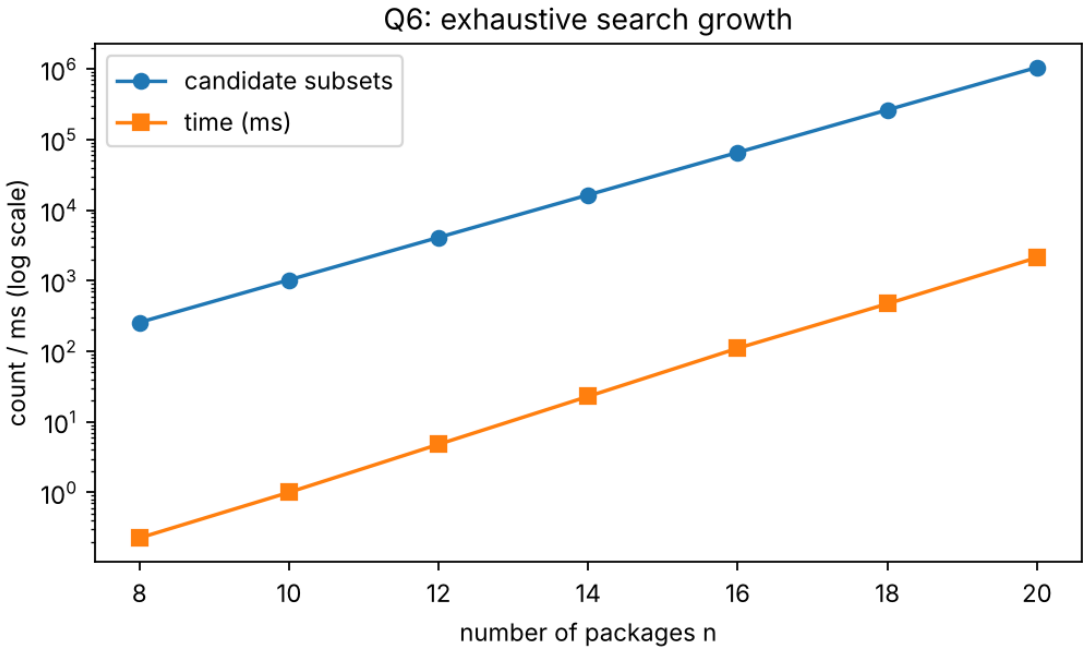


Figure 7. Candidate count and time for increasing n (log scale).

Observed candidate counts equal 2^n exactly for every size. The feasible fraction falls from about 31% at n = 8 to about 22% at n = 20 (228,150 of 1,048,576 at n = 20). Time grew from 469 ms at n = 18 to 2,122 ms at n = 20, a factor of 4.5 for two extra packages, which matches the prediction of about 2 per package.

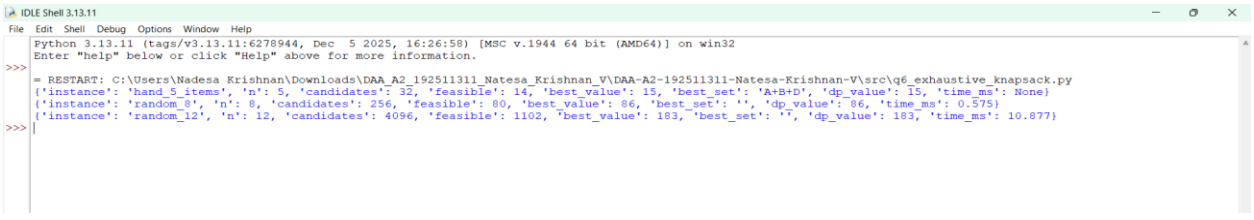
Why a small increase produces a large jump

The solution space is the set of all subsets, and it doubles with every package. The number of packages is an exponent, not a multiplier. Going from 20 to 30 packages multiplies the space by 2^10 = 1,024, even though the input has grown by only 50%. The exhaustive method enumerates this space without pruning, so its work follows the size of the space directly.

Deployment decision

Response budget: I set 1 second for a loading decision. Planners need the answer during a live planning discussion, and anything longer would interrupt that process. Using the measured times, n = 18 takes about 0.47 s and n = 20 about 2.1 s on the reference machine. The largest size I would deploy is n = 18. To leave a safety margin of about 2x for slower hardware, I would limit deployment to n = 17 (about 0.23 s by the same growth rate). Beyond about n = 20, exhaustive search should be replaced by a dynamic-programming or heuristic method, which the dp_check column shows reaches the same optimum for these instances.

Output :



Q7 - Diagnose, Redesign and Validate

Selected solution: the brute-force closest-pair algorithm from Q4. It shows the clearest computational limitation as n grows, and its evidence is already measured.

A. Diagnosis of the computational bottleneck

- Most computational effort goes to distance calculations. The original performs exactly n(n-1)/2 of them: 1,279,200 at n = 1,600, all of which are necessary to be sure of the answer under that method.
- The operation that grows quadratically is the pair comparison itself. Doubling n quadruples the work.
- Repeated work is the distance calculation for pairs that cannot possibly beat the current best. Once the points are sorted by x, a pair whose x-gap already exceeds the best distance so far has no chance of improving the answer, yet the original still computes it.
- The characteristic that causes the limitation is that the algorithm ignores spatial order and examines pairs regardless of how far apart they are.

B. Design objective

Reduce distance calculations by at least 90% at n = 1,600 on uniform random input, while keeping the answer exactly equal to the brute-force answer. The measurable target is a reduction in the number of distance calculations; time is reported as a secondary measure.

C. Revised computational strategy: sort-and-sweep with a pruning rule

Idea: sort points by x once. For each point i, scan later points j in x order. Because the x-gap (x_j - x_i) never decreases along the sorted order, once (x_j - x_i)^2 is at least the best squared distance found so far, no later j can produce a closer pair, so the inner loop can stop. The bound is exact, so no true closest pair can be lost.

- What changed: pairs are no longer all examined. An outer loop over sorted points and a break condition replace the full double loop.
- Property exploited: the x-ordering of the points makes the pruning bound monotone.
- Operations avoided: distance calculations for pairs whose x-gap already exceeds the best distance.
- Preprocessing: one sort by x, costing O(n log n).
- Additional memory: an O(n) sorted copy of the point list.
- Trade-off: the worst case is unchanged. If many points share nearly the same x (for example, a vertical line), the pruning bound never triggers and the algorithm is quadratic again. This limitation is tested below rather than hidden.

```
PRUNED_CLOSEST(P):
  Q <- sort P by (x, y)                  # preprocessing, O(n log n)
  best <- infinity
  for i = 1 .. n:
    for j = i+1 .. n:
      dx <- Q[j].x - Q[i].x
      if dx*dx >= best: break            # no later j can improve
      d2 <- dx*dx + (Q[j].y - Q[i].y)^2  # distance calculation
      if d2 < best: best <- d2; record pair
  return best
```

D. Prediction before implementing

- Distance calculations should fall from n(n-1)/2 to a near-linear count on uniform random points, because the x-window around each point holds only a few candidates once a small best distance has been found.
- The measure expected to improve most is distance calculations; time should improve with them.
- On a vertical-line input, the redesign should give no improvement, since every x-gap is zero and the pruning bound never triggers. This is a deliberate test of the limitation.

E. Implementation and validation

Input	n	Original dist. calcs	Redesign dist. calcs	Original pairs	Redesign pairs	Reduction %	Orig. time ms	Redesign time ms	Same answer
uniform_random	100	4,950	175	4,950	274	96.46	1.056	0.066	True
uniform_random	200	19,900	277	19,900	476	98.61	4.208	0.124	True
uniform_random	400	79,800	545	79,800	944	99.32	19.167	0.259	True
uniform_random	800	319,600	1,589	319,600	2,388	99.5	71.677	0.718	True
uniform_random	1,600	1,279,200	892	1,279,200	2,491	99.93	286.573	0.87	True
vertical_line_stress	200	19,900	19,900	19,900	19,900	0	-	-	True
vertical_line_stress	800	319,600	319,600	319,600	319,600	0	-	-	True

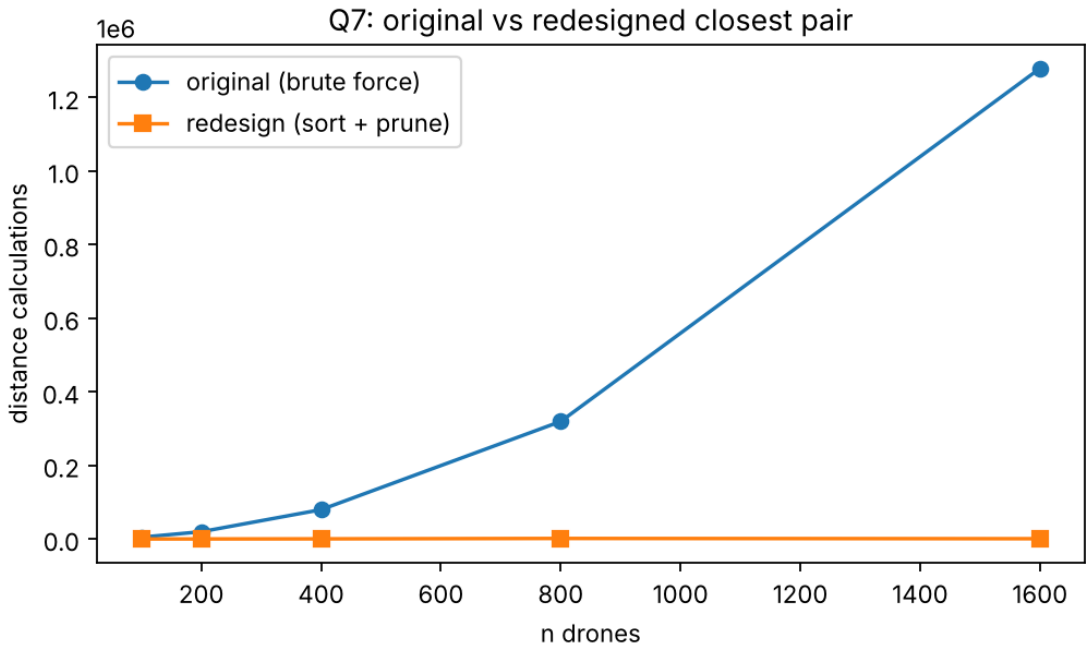


Figure 8. Distance calculations: original (brute force) and redesigned, uniform random input.

Every redesigned answer equals the brute-force answer, as the correctness requirement demands. On uniform random input the reduction is 96.5% at n = 100, 99.3% at n = 400 and 99.9% at n = 1,600. Redesign time was 0.87 ms at n = 1,600 against 286.6 ms for the original, a speed-up of about 330 times, and includes the sort. On the vertical-line stress input, the reduction is 0% and the pair counts are identical, as predicted.

F. Defence of the final design

- Prediction support: the prediction was supported on uniform random input, with near-linear behaviour. The pairs-examined count grew from 274 at n = 100 to 2,491 at n = 1,600, a factor of 9.1 for 16 times more points, roughly n^0.8 over this range. The vertical-line prediction was also supported.
- Magnitude: 96% to 99.9% fewer distance calculations and roughly 16 to 330 times faster in the measured range. The gain grows with n, which is what the diagnosis predicted.
- New costs and trade-offs: the sort adds O(n log n) work and O(n) memory, and the worst-case complexity remains quadratic for inputs with many equal x values.
- When the original is still preferable: for very small n (for example n = 10 or 20), the brute-force code is simpler to verify and the difference in time is negligible. It is also a useful reference oracle for testing the redesign, as done here.
- Sufficiency: the redesign addresses the diagnosed limitation for typical spatial spreads, but not for adversarial geometry. A stronger guarantee would need a different strategy, such as divide-and-conquer or grid bucketing; I have not implemented those here, since the assignment asks for a designed strategy rather than a textbook replacement.

Conclusion: the redesigned sort-and-sweep method substantially removes the quadratic distance-calculation cost for typical drone layouts. It is an engineering improvement with a clearly stated limit, and I would keep brute force as a test oracle and for very small fleets.

Output :

